Problem A. 哇，昨日六次重现

这是一道交互题。

继 2018 至 2024 年成功承办赛事之后，南京航空航天大学（NUAA）将连续第八年承办国际大学生程序设计竞赛（ICPC）。

在 2018 与 2019 年，“中二之力” 队与 “三个顶俩” 队为清华大学赢得了冠军。在 2020，2021 与 2022 年，北京大学的 “逆十字” 队赢得三连冠。在 2023 年，来自北京大学的另一支队伍 “重生之我是菜狗” 赢得了冠军。在 2024 年，浙江大学的 “来生” 队摘得桂冠。

今年，将会有约 335 支队伍参与南京站的竞赛。本次竞赛将会颁发至多 33 项金奖，66 项银奖与 99 项铜奖（数字仅供参考）。让我们期待选手们出色的表现！我们还想要感谢竞赛组委会与志愿者们的努力付出。感谢你们为本次竞赛做出的贡献！



在 2023 ICPC 国际大学生程序设计竞赛亚洲区域赛（南京站）中拍摄的照片

在 2018 年的竞赛中，K 题《袋鼠谜题》要求选手为以下游戏构造一个操作序列：

> 谜题由一个 n 行 m 列的网格（$1 \le n, m \le 20$）组成，且有一些（至少 2 只）袋鼠位于网格中。玩家的目标是控制袋鼠并把它们聚集在同一个格子中。一些格子里有墙，袋鼠无法进入这些有墙的格子，而其它格子是空的。袋鼠可以从一个空格子移动到上，下，左，右相邻的另一个空格子中。
>
> 游戏开始时，每个空格子里都有一只袋鼠。玩家可以通过键盘上 U，D，L，R 四个按键控制袋鼠的移动。所有袋鼠会同时根据您按下的按键移动。
>
> 选手需要构造一个长度至多为 5×10^4 且由 U，D，L，R 组成的操作序列以达成目标。

在 2020 年的竞赛中，A 题《啊，昨日重现》要求选手构造一张输入地图，以证明以下代码并不是上述问题的解：

```
#include <bits/stdc++.h>
using namespace std;
string s = "UDLR";
int main()
{
  srand(time(NULL));
  for (int i = 1; i <= 50000; i++) putchar(s[rand() % 4]);
  return 0;
}
```

此外，在 2021 年的竞赛（A 题，《呀，昨日再次重现》），2022 年的竞赛（A 题，《停停，昨日请不要再重现》）2023 年的竞赛（A 题，《酷，昨日四次重现》），和 2024 年的竞赛（A 题，《嘿，有看到我的袋鼠吗?》）中，每年都有一道与袋鼠相关的问题！我们很想向您介绍所有这些问题，但如果我们每年都这样做，在 3025 年的竞赛中将会有一道题拥有长达 500 页的题面。因此，这次我们省略它们。另外，您可能已经在热身赛中见过它们了。

在 2025 年的竞赛中，如大家期待的那样，袋鼠题又回来啦！我们不知道为什么命题组的成员们那么喜欢袋鼠，但题目如下：

给定一张 n 行 m 列的网格，在位于第 i_h 行第 j_h 列的格子上有一个洞，其它每个格子都是空地并且都有一只袋鼠。

相似地，袋鼠可以被键盘上的 U，D，L，R 键控制。所有袋鼠会同时根据按下的按键移动。具体来说，对于一只位于第 i 行第 j 列的格子（用 (i, j) 表示）上的袋鼠：

1. 按键 U：它会移动到 $(i-1, j)$。
2. 按键 D：它会移动到 $(i+1, j)$。
3. 按键 L：它会移动到 $(i, j-1)$。
4. 按键 R：它会移动到 $(i, j+1)$。

如果一只袋鼠踩到了洞（也就是说，$i = i_h$ 且 $j = j_h$）或者移动到了网格外面，它将被从网格上移除。

问题在于，i_h 与 j_h 的值是未知的。您需要在至多 $(n + m + 10)$ 次询问内确定洞的位置。每次询问，您可以按一个按键（U，D，L，或 R）。裁判程序会输出一个整数 t 作为询问的答案，表示在按键后，网格上还剩几只袋鼠。

请注意：裁判程序不是适应性的。也就是说，每组测试数据的答案是事先确定的。

Input

有多组测试数据。第一行输入一个整数 T（$1 \le T \le 100$）表示测试数据组数，对于每组测试数据：

第一行输入两个整数 n 和 m（$3 \le n, m \le 30$），表示网格的行数和列数。

Interaction Protocol

要提出询问，请输出一行。首先输出 `?`，之后跟一个空格，然后输出一个英文字母（U，D，L，或 R）。在清空输出缓冲区之后，您的程序需要读入一个整数 t，表示对您的询问的回答。请回忆：对于每组数据，您至多可以提出 $(n + m + 10)$ 次询问。

要猜测洞的位置，请输出一行。首先输出 `!`，之后跟一个空格，然后输出两个由单个空格分隔的整数 i_h 和 j_h（$1 \le i_h \le n, 1 \le j_h \le m$）表示洞的位置。在清空输出缓冲区之后，您的程序应该马上开始处理下一组测试数据。如果没有更多测试数据，您的程序应该立即退出。还请注意，猜测不算一次询问。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)`（如果您使用 `printf`）或 `cout.flush()`（如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()`。
- Python 使用 `stdout.flush()`。

Example

standard input	standard output
2 3 4 7 3 4 3 8 5	 ? L ? D ! 2 3 ? U ? R ! 1 1